B)
The first algorithm has exponential time complexity because each function call makes three recursive calls. As n inputs grow the number of recursive calls grows exponentially because each call generates additional calls. The second algorithm has linear time complexity since it only makes one recursive call for each function call. It keeps track of the 3 previous oddonacci numbers to avoid recalculating a number multiple times. The second algorithm fixed the bottleneck when calculating the oddonacci number at a position larger than 15 by not recalculating a number multiple times since this is the reason the first algorithm gets slower as the numbers get larger.

C)
No, both my algorithms did not use tail recursion since they both needed to add up the numbers after all the recursive steps were done. Both algorithms calculate the 3 previous oddonacci numbers then add them up to calculate the number that was requested and for that reason they cannot be tail recursive.

D)
The first difference between my code and the ai's code is that I created a class for the oddonacci result and made the linear function return an object of that class. The ai passed the last 3 numbers as a parameter in a helper function.The second difference is my code prompts a user for an index then calculates the oddnacci number to that index while the ai's code just loops through all oddonacci numbers from 5 to 200 in increments of 5. This does make testing easier since no input is required from the user, however my code can be used to calculate any number not only increments of 5. The ai made a mistake while creating the program because it outputs everything to the console even though it was specified to create a text file. The code created by the ai would be more efficient for testing which function is more time efficient since it does not require input however if we are calculating just one number and we want to know the time efficiency for a specific number not only increments of 5 then mine would be more accurate.

Prompt:
Attached file to provide assignment details
Message attached : only do part d dont answer the question just make a implementation that would work in java. Follow guidelines given

AI CODE:

```
/*
 * COMP 352 - Fall 2026
 * Programming Assignment 1
 * Part D - AI Generated Implementation
 *
 * This program contains two recursive implementations of Oddonacci:
 * 1. Multiple recursive version
```

```
 * 2. Linear recursive version
 *
 * Only simple Java features are used:
 * - Classes
 * - Methods
 * - Primitive variables
 * - Arrays
 * - Recursion
 * - Loops
 *
 * No advanced data structures are used.
 */

public class OddonacciAI {

    /*
     * Multiple recursive Oddonacci.
     *
     * The function recursively calculates the three preceding
     * Oddonacci values and adds them together.
     *
     * Base cases:
     * O(1) = 1
     * O(2) = 1
     * O(3) = 1
     */
    public static long multipleOddonacci(int n) {

        if (n == 1 || n == 2 || n == 3) {
            return 1;
        }

        return multipleOddonacci(n - 1)
                + multipleOddonacci(n - 2)
                + multipleOddonacci(n - 3);
    }

    /*
     * Helper method for the linear recursive version.
     *
     * Instead of recalculating the same Oddonacci values,
     * this method carries the three most recent values with it.
     *
     * a = O(n - 2)
```

```
 * b = O(n - 1)
 * c = O(n)
 *
 * When n reaches the requested position, c is returned.
 */
public static long linearHelper(int n, long a, long b, long c) {

    if (n == 3) {
        return c;
    }

    return linearHelper(n - 1, b, c, a + b + c);
}

/*
 * Linear recursive Oddonacci.
 *
 * The first three Oddonacci values are all 1.
 */
public static long linearOddonacci(int n) {

    if (n == 1 || n == 2 || n == 3) {
        return 1;
    }

    return linearHelper(n, 1, 1, 1);
}

/*
 * Measures the running time of the multiple recursive version.
 *
 * System.nanoTime() is used because it provides a finer
 * measurement than System.currentTimeMillis().
 */
public static long measureMultiple(int n) {

    long start = System.nanoTime();

    long result = multipleOddonacci(n);

    long end = System.nanoTime();

    // Print result so that the calculation is actually performed.
    System.out.println("Multiple Oddonacci(" + n + ") = " + result);
```

```
    return end - start;
}

/*
 * Measures the running time of the linear recursive version.
 */
public static long measureLinear(int n) {

    long start = System.nanoTime();

    long result = linearOddonacci(n);

    long end = System.nanoTime();

    // Print result so that the calculation is actually performed.
    System.out.println("Linear Oddonacci(" + n + ") = " + result);

    return end - start;
}

/*
 * Main method.
 *
 * Calculates Oddonacci values for:
 * 5, 10, 15, ..., 200
 *
 * The program prints the result and execution time for
 * each version.
 */
public static void main(String[] args) {

    System.out.println("================================================");
    System.out.println("COMP 352 - Oddonacci Runtime Experiment");
    System.out.println("AI Generated Version - Part D");
    System.out.println("================================================");

    System.out.println();
    System.out.println("MULTIPLE RECURSIVE VERSION");
    System.out.println("------------------------------------------------");

    for (int n = 5; n <= 200; n += 5) {

        /*
```

```
         * The multiple recursive algorithm becomes extremely
         * slow as n increases. Therefore, very large values
         * may take a long time to finish.
         */
        long start = System.nanoTime();

        long result = multipleOddonacci(n);

        long end = System.nanoTime();

        long elapsed = end - start;

        System.out.println(
                "n = " + n
                + ", Oddonacci = " + result
                + ", Time = " + elapsed + " ns"
        );
    }

    System.out.println();
    System.out.println("LINEAR RECURSIVE VERSION");
    System.out.println("-----------------------------------------------");

    for (int n = 5; n <= 200; n += 5) {

        long start = System.nanoTime();

        long result = linearOddonacci(n);

        long end = System.nanoTime();

        long elapsed = end - start;

        System.out.println(
                "n = " + n
                + ", Oddonacci = " + result
                + ", Time = " + elapsed + " ns"
        );
    }

    System.out.println();
    System.out.println("Experiment completed.");
  }
}
```